

Strategic Architectural Analysis for the Migration of TxDOT's GRIB2 Data Processing Pipeline

Executive Summary

This report presents a comprehensive feasibility analysis and strategic recommendation for the Texas Department of Transportation (TxDOT) concerning the modernization of its GRIB2 meteorological data processing pipeline. The primary objective is to evaluate the viability of migrating the current workflow, which operates in a flexible but operationally intensive Amazon EC2 environment, to a fully managed, native Snowflake architecture. The analysis culminates in a fact-based comparison of architectural patterns to determine the optimal future state for TxDOT's data infrastructure.

The core technical challenge of this workload lies not in data volume, but in dependency complexity. The GRIB2 data format necessitates the use of highly specialized software: the `pygrib` Python library and the `wgrib2` command-line utility. Both of these tools have deep-seated dependencies on compiled C and Fortran libraries, most notably the `eccodes` library from the European Centre for Medium-Range Weather Forecasts (ECMWF). This requirement for a specific, customizable, and often complex build environment is the central issue against which all potential architectures must be measured.

This report evaluates three distinct architectural models:

1. **The Incumbent EC2-Centric Model:** The current state, offering maximum software flexibility at the cost of high operational overhead.
2. **The "Fully Native" Snowflake Model:** A visionary state where all processing occurs within Snowflake's Snowpark environment, promising a serverless, fully managed experience.
3. **The Recommended Hybrid Cloud Model:** A pragmatic approach that combines the strengths of AWS specialized compute with Snowflake's data platform capabilities.

The key findings of this analysis are as follows:

- A **"Fully Native" Snowflake architecture is not feasible** for the complete end-to-end GRIB2 processing workflow. The secure, sandboxed nature of the Snowpark environment, a key feature of the Snowflake platform, fundamentally prohibits the installation of the required compiled C/Fortran libraries and the execution of external command-line utilities like `wgrib2`. Attempts to install the necessary dependencies via Snowflake's provided mechanisms carry an unacceptably high risk of failure and are not

supported for this level of complexity.

- The incumbent EC2 model, while functionally capable, represents a legacy approach to cloud architecture. It saddles TxDOT with significant and ongoing responsibilities for infrastructure management, security patching, manual scaling, and software maintenance, leading to a higher Total Cost of Ownership (TCO) when accounting for operational effort.
- A **Hybrid Cloud Architecture**, specifically one that utilizes an external orchestrator like Amazon Managed Workflows for Apache Airflow (MWAA) to coordinate processing on dedicated, ephemeral compute resources (such as AWS Fargate or EC2) with data storage and analytics in Snowflake, emerges as the optimal path forward.

Therefore, this report formally recommends that TxDOT adopt the **Orchestrated Compute Hybrid Architecture**. This model strategically leverages the unique strengths of each platform: the unparalleled flexibility of AWS for specialized, compiled-code execution and the unmatched power of Snowflake for scalable data storage, governance, and analytics. This approach mitigates the risks and drawbacks of the other models, ensuring technical reliability, superior scalability, and the lowest long-term TCO for TxDOT's critical meteorological data pipelines.

Deconstructing the GRIB2 Processing Challenge

To accurately assess the feasibility of any architectural pattern, it is imperative to first establish a granular understanding of the technical requirements inherent in the data and the tools used to process it. The GRIB2 workflow is not a conventional data engineering task; it is a scientific computing problem with a unique and demanding set of dependencies.

The GRIB2 Data Format: A Technical Primer

GRIB (GRIdded Binary) is the standard format established by the World Meteorological Organization for the storage and transport of historical and forecast weather data. The version in question, GRIB2, is a highly efficient, compressed binary format. Its design prioritizes compact file sizes for transmitting massive datasets, not human readability or simple parsing.

A GRIB2 file is not a monolithic data grid but rather a collection of individual records. Each record contains a complex set of metadata headers describing the data's parameters (e.g., temperature, pressure, wind speed), projection, time, and vertical level, followed by the grid of data values themselves. To extract meaningful information, one cannot simply read the file; it must be decoded using specialized software that understands the intricate GRIB2 specification. This fundamental characteristic dictates the need for specific, non-standard tooling.

Analysis of Core Processing Tools and Their Dependencies

The TxDOT workflow relies on a standard toolkit within the meteorology domain for processing GRIB2 files. The feasibility of any migration hinges entirely on the target platform's ability to support this specific software stack.

The pygrib Python Library

The pygrib library serves as a primary Python interface for reading and interacting with GRIB files.¹ It allows developers to open GRIB files, list the messages they contain, and extract both metadata and data arrays for use in Python applications, often with libraries like numpy.

However, pygrib is not a self-contained, pure-Python library. It is a thin wrapper that provides Python bindings for a powerful underlying C library.

- **Critical Dependency:** The functionality of pygrib is entirely dependent on the **eccodes C library** developed by the European Centre for Medium-Range Weather Forecasts (ECMWF).²

eccodes is the engine that performs the actual decoding of the GRIB format.

- **Installation Complexity:** While a simple pip install pygrib or conda install pygrib often works on a developer's local machine, this is because the package managers are abstracting away a complex dependency resolution process.¹ This process involves downloading a pre-compiled binary of the eccodes library or, in some cases, compiling it from source. On certain Linux distributions, this requires the pre-installation of system-level development packages like libeccodes-dev or python3-grib using a package manager like apt, which in turn installs the necessary C libraries.³ This deep-seated reliance on a compiled, non-Python, system-level library is the central challenge for any managed, sandboxed execution environment.

The wgrib2 Command-Line Utility

The wgrib2 utility is a versatile and powerful standalone tool for manipulating GRIB2 files directly from the command line.⁴ It is widely used for tasks such as creating inventories of GRIB files, extracting specific records based on matching criteria, regriding data, and performing simple calculations without needing to write a full application.

The challenge with wgrib2 is its installation, which is significantly more involved than a typical software package.

- **Complex Compilation from Source:** wgrib2 is written in C and Fortran and must be compiled from source for the target operating system.⁵ This process requires a full

development toolchain, including C (gcc) and Fortran (gfortran) compilers.⁶

- **Multiple System-Level Dependencies:** The compilation process requires linking against several other specialized scientific libraries, which must be present on the system. These often include libraries for NetCDF, PNG (for compression), and potentially Jasper or OpenJPEG for handling JPEG2000 compression schemes used in some GRIB2 records.⁵
- **Python Interoperability:** While wgrib2 can be compiled into a shared library (libwgrib2.so on Linux) that can be called from Python, this is an advanced procedure that underscores the need for complete control over the build environment and linking process.⁵

The following table provides a clear, consolidated map of this dependency chain, which will serve as the technical foundation for evaluating the architectural options.

Tool	Primary Language	Key Python-Level Dependencies	Key Compiled/System-Level Dependencies	Typical Installation Method
pygrib	Python (Wrapper)	numpy, pyproj ²	eccodes C library ²	pip install, conda install ¹
wgrib2	C / Fortran	N/A	gcc, gfortran, NetCDF libs, Jasper/OpenJPEG libs ⁵	Compile from source (make) ⁵

The implications of this dependency map are profound. The GRIB2 processing workflow cannot be treated as a generic ETL task that can be accomplished with standard, pure-Python libraries or SQL. The success of any proposed architecture is not contingent on its ability to handle large data volumes or execute parallel transformations, but on its fundamental ability to accommodate a custom, compiled, scientific software environment. This reframes the evaluation from a "Big Data" problem to a "Complex Dependency Management" problem, a critical distinction that will guide the remainder of this analysis.

Architectural Deep Dive 1: The Incumbent EC2-Centric Model

The current architecture employed by TxDOT is a common and well-understood pattern for handling workloads with specialized software requirements. It leverages the infrastructure-as-a-service (IaaS) offering of Amazon Web Services (AWS), specifically EC2

instances, to create a bespoke processing environment.

Capabilities and Execution Model

The defining characteristic of the EC2-centric model is the complete control it affords over the underlying operating system. This control is the key enabler for the GRIB2 workflow.

- **Unrestricted System Access:** TxDOT has administrative (root or sudo) privileges on the EC2 instances.⁸ This level of access is the prerequisite for installing the complex dependencies required by the GRIB2 tooling.
- **Unrestricted Software Installation:** With administrative access, engineers can use system-level package managers (e.g., yum on Amazon Linux, apt-get on Ubuntu) to install the entire development toolchain, including gcc, gfortran, and critical development libraries like libeccodes-dev and libnetcdf-dev.³
- **Direct Compilation and Configuration:** This environment permits the direct compilation of software like wgrib2 from its source code. This provides granular control over build flags, library versions, and installation paths, ensuring that the utility is perfectly tailored to the system and the specific needs of the data pipeline.⁵ Similarly, pygrib and its eccodes dependency can be installed with confidence, as all underlying system requirements can be met beforehand.
- **Flexible Orchestration:** The pipeline logic can be implemented and scheduled using a wide array of tools. This can range from simple cron jobs running shell scripts that execute wgrib2 commands, to more sophisticated Python applications that use the subprocess module to call wgrib2 and the pygrib library for in-memory data manipulation.

Data Flow and Integration with Snowflake

In this model, the EC2 instance acts as a dedicated, external processing node in the broader data ecosystem. The typical data flow is as follows:

1. Raw GRIB2 files are ingested from a source and land on the EC2 instance's local storage (Elastic Block Store - EBS) or are downloaded from an S3 bucket.
2. An orchestration script on the EC2 instance executes the processing logic, using wgrib2 and/or pygrib to decode the binary GRIB2 data into a structured, tabular format like CSV or, more efficiently, Apache Parquet.
3. The newly created structured files are then uploaded to a designated "processed" S3 bucket.
4. This S3 event triggers a data load into Snowflake. This is typically accomplished using Snowpipe, Snowflake's continuous data ingestion service. A pipe is configured to watch the S3 bucket, and upon the creation of a new file, it automatically ingests the data into a target Snowflake table.¹¹ This creates a seamless handoff from the external processing

environment to the centralized data platform.

Assessment of Strengths and Weaknesses

While functionally effective, the EC2-centric model presents a significant trade-off between flexibility and operational cost.

Strengths:

- **Maximum Flexibility and Guaranteed Compatibility:** This is the paramount advantage. The ability to control the entire software stack from the OS up guarantees that any dependency, no matter how complex or obscure, can be installed and configured. This completely de-risks the technical feasibility of the processing step.
- **Mature and Proven Pattern:** This is a traditional and well-understood approach for integrating specialized workloads with data warehouses. The patterns and tools are mature and widely documented.

Weaknesses:

- **High Operational Overhead:** This is the model's primary drawback. In stark contrast to the fully managed nature of the Snowflake platform ¹³, the EC2 model places the full burden of infrastructure management on TxDOT's engineering team. This includes OS patching, applying security updates, managing user access, monitoring instance health, and handling software upgrades.
- **Inefficient Scalability and Cost Model:** Scaling the processing capacity requires manual intervention or the configuration and management of AWS Auto Scaling Groups. Handling unpredictable or spiky workloads is challenging; it often leads to either persistent over-provisioning of instances to meet peak demand (incurring unnecessary cost) or under-provisioning, which creates performance bottlenecks and delays in data availability. The cost model is tied to instance-hours, which can be inefficient if the pipeline runs infrequently.
- **Significant Total Cost of Ownership (TCO):** The TCO of this model extends far beyond the direct cost of EC2 compute hours. It must include the significant "soft costs" of the engineering and system administration time dedicated to maintaining the infrastructure, which could otherwise be spent on higher-value activities.
- **Data Governance and Lineage Gaps:** The processing logic and its environment are completely isolated from the Snowflake data platform. This creates a "black box" in the data lineage. While the final load into Snowflake can be tracked, the transformations that occurred on the EC2 instance are opaque to Snowflake's governance and metadata management tools, making end-to-end auditing and impact analysis more difficult.

Architectural Deep Dive 2: The "Fully Native" Snowflake Model

The "fully native" model represents the ideal state from a platform consolidation perspective. The vision is to eliminate the external EC2 processing node entirely and perform all data transformation steps within the Snowflake ecosystem. This would leverage Snowpark, Snowflake's framework for running non-SQL code, such as Python, directly within the platform's compute engine.¹⁴

The Vision: Snowpark as the Sole Processing Engine

In this envisioned architecture, the GRIB2 pipeline would be re-implemented as a Snowpark Python User-Defined Function (UDF) or Stored Procedure.¹⁶ The data flow would be radically simplified:

1. Raw GRIB2 files from the source would be loaded directly into a Snowflake internal stage using an automated Snowpipe.¹¹
2. A Snowpark Python UDF would be triggered. This UDF would be designed to read the raw binary GRIB2 file directly from the stage.
3. The Python code within the UDF would perform the decoding and transformation logic, leveraging the required libraries.
4. The UDF would output structured data, which would be written directly into a target Snowflake table.

This approach promises to leverage the core benefits of the Snowflake platform: a fully managed, serverless execution model where Snowflake handles all compute provisioning, scaling, and maintenance, allowing TxDOT to focus solely on the business logic of the pipeline.¹³

The Dependency Gauntlet: A Fact-Based Feasibility Test for pygrib

The viability of this entire vision rests on a single, critical question: can the pygrib library and its fundamental eccodes C-library dependency be successfully installed and run within the Snowpark Python sandbox? Snowflake provides several mechanisms for incorporating third-party Python packages into UDFs.¹⁸ We will now systematically evaluate each path against the specific requirements of pygrib.

Path 1: The Anaconda Channel

Snowflake has a partnership with Anaconda to provide a curated, secure channel of pre-vetted Python packages that can be seamlessly used in Snowpark functions.¹⁶ When a package is specified from this channel, Snowflake automatically installs it and its

dependencies within the sandboxed warehouse environment.¹⁸

- **Analysis:** An examination of the list of packages available in the official Snowflake Anaconda channel reveals that neither pygrib nor its underlying dependency eccodes are included.²⁰
- **Verdict: Not Feasible.** This primary and most straightforward path for dependency management is not available for the required GRIB2 tooling.

Path 2: The PyPI Artifact Repository

A more recent and powerful feature is Snowpark's direct support for packages from the public Python Package Index (PyPI).²¹ This allows developers to specify any package from PyPI, and Snowflake will attempt to download and install it. However, this feature comes with a critical limitation: it currently only supports packages distributed as binary wheel (.whl) files.¹⁸

- **Analysis:** The pygrib project *does* publish pre-compiled wheel files to PyPI for common Linux distributions, specifically targeting the manylinux_2_17_x86_64 platform.¹ This appears, on the surface, to be a promising avenue. However, the success of this installation is entirely contingent on the underlying operating system environment of the Snowpark sandbox. A manylinux wheel is built against a specific version of core system libraries (like glibc). While designed for broad compatibility, there is no guarantee that the specific versions of all system libraries that the compiled eccodes C-code within the wheel depends on are present and accessible within Snowflake's highly restricted and proprietary sandbox. Snowflake's own documentation highlights that UDFs run in a restricted engine with security constraints that disable certain capabilities.²² It is highly improbable that a complex scientific library requiring specific C-library versions would operate reliably in this opaque environment.
- **Verdict: High Risk and Functionally Infeasible.** While theoretically possible if every unknown condition is met, relying on this path introduces a critical, untestable dependency on the internal, undocumented implementation details of Snowflake's sandbox. This is not a robust foundation for a mission-critical production pipeline. Any silent change in Snowflake's underlying infrastructure could break the pipeline without warning.

Path 3: Custom Staged Packages

The final option allows users to package their own Python code and its dependencies into a .zip file, upload it to a Snowflake stage, and reference it in the UDF definition.¹⁶ The snow snowpark package command-line tool can assist in creating these packages.¹⁹

- **Analysis:** This mechanism is designed for bundling pure-Python modules or vendoring

other Python packages. It is fundamentally incapable of packaging and deploying the required system-level dependencies. The process zips up .py files and their Python-level dependencies; it cannot capture and reinstall the compiled libeccodes.so C-library or the system toolchain needed to build it. The sandbox environment where the UDF runs would still lack the core C library that pygrib needs to function.

- **Verdict: Not Feasible.** This path is unsuitable for packages with compiled, non-Python dependencies.

The wgrib2 Impasse: A Hard No for Command-Line Utilities

The analysis for the wgrib2 utility is far more straightforward. Snowpark UDFs execute Python code within a secure, isolated, and hermetically sealed Python runtime environment.²² This is a core design principle of the platform to ensure security, stability, and multi-tenant safety.

- **Analysis:** There is absolutely no mechanism within the Snowpark UDF framework to execute arbitrary shell commands, external binaries, or compiled programs like wgrib2. The UDF cannot access the underlying file system, call a subprocess, or otherwise break out of its Python sandbox.
- **Verdict: Categorically Infeasible.** If the TxDOT workflow has a hard dependency on the wgrib2 utility for any processing step, a fully native Snowflake architecture is impossible by design.

Report Card: What Snowflake Can and Cannot Do Natively for GRIB2 Processing

The detailed analysis leads to a clear and unambiguous conclusion about the capabilities of a native Snowflake architecture for this specific use case.

Capability	Feasibility in Native Snowflake	Rationale
Ingest Raw GRIB2 Files	Can Do	Snowpipe can ingest any file type, including binary GRIB2, into a Snowflake stage. ¹¹
Store Raw GRIB2 Files	Can Do	Files can be stored in an internal stage or loaded into a table column of type BINARY.
Install pygrib Library	Cannot Do (Reliably)	The critical eccodes C-library dependency cannot be reliably installed in the sandboxed environment. ¹⁸
Execute wgrib2 Utility	Cannot Do	Snowpark UDFs are sandboxed and cannot execute external

		command-line binaries. ²³
Process Structured Data	Can Do (Excellently)	Once data is in a structured format (e.g., Parquet, CSV), Snowpark and SQL are exceptionally powerful for transformation and analysis. ¹⁴

The attempt to force a "fully native" Snowflake solution for this workflow fails not because of any deficiency in Snowflake's data processing capabilities, but because of a fundamental architectural mismatch. The value proposition of Snowflake is a secure, fully managed, and isolated platform-as-a-service (PaaS).¹³ This requires sandboxing user code. The GRIB2 tooling, conversely, requires an open, customizable infrastructure-as-a-service (IaaS) environment with deep system-level access. These two design philosophies are diametrically opposed. Therefore, the architectural goal for TxDOT should not be to force an incompatible workload onto a platform, but to design a solution that uses each platform for its intended purpose.

Architectural Deep Dive 3: The Optimal Path - Hybrid Cloud Patterns

Given the infeasibility of a fully native Snowflake solution and the operational drawbacks of the incumbent EC2 model, the optimal path lies in a hybrid architecture. A hybrid model does not treat platforms as mutually exclusive but instead composes a solution that leverages the best-in-class capabilities of each. Snowflake itself anticipates and supports these patterns through features like External Functions, acknowledging that certain specialized logic is best executed outside the core data platform.²⁵ This section evaluates two pragmatic hybrid patterns.

Pattern A: The External Function Architecture (Leveraging AWS Lambda)

This pattern maintains a tight, synchronous integration with Snowflake's SQL interface while outsourcing the specialized computation to a serverless function.

Architectural Blueprint

1. **Ingestion:** A process (e.g., a script, a data transfer service) deposits raw GRIB2 files into a designated AWS S3 bucket.
2. **Metadata Load:** A Snowflake Snowpipe is configured to watch this S3 bucket. Upon the

arrival of a new file, the pipe ingests only the file's metadata—specifically its S3 path—into a Snowflake staging table.

3. **External Function Call:** A Snowflake task or a downstream process executes a SQL statement that calls a custom **External Function** for each new file path recorded in the staging table.²⁵
4. **API Gateway & Lambda Trigger:** The Snowflake External Function is configured to call an AWS API Gateway endpoint. This endpoint acts as a secure proxy that, in turn, triggers an AWS Lambda function, passing the S3 file path in the request payload.²⁵
5. **Lambda Processing:** The core GRIB2 processing logic resides within the AWS Lambda function. The pygrib library and its eccodes C-library dependency are packaged into a **Lambda Layer**, a mechanism for sharing common dependencies across functions. This is a complex but well-defined, one-time setup task. The Lambda function uses the provided S3 path to read the GRIB2 file, decodes it using pygrib, and formats the output as a JSON structure.
6. **Data Return:** The Lambda function returns the structured JSON data in its response. This response travels back through API Gateway to the calling Snowflake External Function.²⁶
7. **Final Load:** The Snowflake SQL statement that initiated the call receives the JSON payload as a VARIANT type and can directly INSERT it into a final, structured destination table.

Assessment

- **Pros:** This architecture is fully serverless, eliminating the need to manage any EC2 instances. It scales automatically with demand, as both Lambda and Snowflake handle compute provisioning. The integration feels "native" to a SQL developer, as the entire process is kicked off by a SQL call within Snowflake.
- **Cons:** This pattern is constrained by the inherent limitations of the AWS Lambda platform. Lambda functions have a maximum execution timeout of 15 minutes and a synchronous invocation payload size limit of 6MB. If a single GRIB2 file is exceptionally large or requires processing that exceeds these limits, the function will fail. This makes the pattern potentially brittle for workloads with highly variable file sizes and complexity.

Pattern B: The Orchestrated Compute Architecture (Leveraging MWAA/EC2/Fargate)

This is a more robust, powerful, and flexible hybrid pattern that uses a dedicated orchestrator to manage a classic, asynchronous Extract-Transform-Load (ETL) workflow.

Architectural Blueprint

1. **Orchestration Hub:** The entire pipeline is managed by an orchestration service, with Amazon Managed Workflows for Apache Airflow (MWAA) being an excellent cloud-native choice.²⁹
2. **Trigger:** The Airflow pipeline (a Directed Acyclic Graph, or DAG) is triggered by a sensor that detects the arrival of a new GRIB2 file in an S3 bucket.
3. **Task 1: Specialized Processing:** The first task in the DAG executes the GRIB2 processing job. This job runs in a dedicated, purpose-built compute environment. This could be:
 - A transient EC2 instance launched for the job and terminated upon completion.
 - An AWS Fargate container, which provides a serverless container execution environment. This is often the preferred modern approach.
 - The environment (e.g., the Docker image for Fargate) is pre-built with the exact versions of wgrib2, pygrib, eccodes, and all other necessary system-level dependencies. This task reads the raw GRIB2 file from the source S3 bucket, decodes it, and writes the structured output (ideally in a highly-efficient columnar format like Apache Parquet) to a separate "processed" S3 bucket.
4. **Task 2: Snowflake Ingestion:** Upon the successful completion of the processing task, the Airflow DAG proceeds to the next task. This task connects to Snowflake and triggers the data load. This can be done in two ways:
 - Executing a COPY INTO <table> command to bulk-load the newly created Parquet file(s) from the processed S3 bucket.
 - Calling the SYSTEM\$PIPE_FORCE_RESUME() function to trigger an existing Snowpipe to check for new files and ingest them.The integration between MWAA and Snowflake is well-supported, often using the Snowflake provider for Airflow.²⁹

Assessment

- **Pros:** This is the most powerful, flexible, and reliable architecture. It imposes **no limitations** on the processing logic; any tool, library, or custom binary can be baked into the compute environment. There are no practical limits on execution time, memory, or CPU, as the compute environment can be sized appropriately for the workload. Using ephemeral compute (like Fargate or transient EC2) ensures that costs are incurred only during active processing. It cleanly separates the concern of specialized computation from the concern of data warehousing.
- **Cons:** This pattern introduces an additional component to the architecture: the orchestrator (Airflow). This adds a degree of complexity compared to the External Function pattern, as it requires the development and maintenance of Airflow DAGs. However, this is a standard and highly valuable pattern for modern data engineering.

Comparative Analysis and Strategic Recommendation for TxDOT

The final step is to synthesize the findings from the architectural deep dives into a direct comparison, leading to a conclusive and actionable recommendation for TxDOT. The choice of architecture will have long-term implications for cost, maintainability, scalability, and risk.

Architectural Comparison Matrix

The following matrix provides a consolidated, at-a-glance comparison of the three primary architectural models across key business and technical criteria.

Criterion	Incumbent EC2-Centric	"Fully Native" Snowflake	Hybrid (Orchestrated Compute)
Technical Feasibility (GRIB2)	Guaranteed	Not Feasible	Guaranteed
Scalability	Manual / High Effort	Automatic (but infeasible)	Automatic / Managed
Infrastructure Mgt. Overhead	High	None	Low
Compute Cost Model	Fixed / Instance-Hour	Consumption-Based	Ephemeral / Consumption-Based
Long-Term Maintainability	Low (brittle, manual)	N/A	High (declarative, modular)
Data Governance & Lineage	Poor (siloe processing)	Excellent (unified)	Good (orchestrated handoff)
Development Agility	Medium	Low (restrictive)	High (modular, best-of-breed)
Overall Risk Profile	Medium (operational risk)	High (technical feasibility risk)	Low (proven, robust pattern)

In-Depth Comparison

- **Technical Feasibility and Reliability:** The analysis has definitively shown that the

"Fully Native" Snowflake model is not technically feasible due to its inability to support the required compiled software stack.¹⁸ This eliminates it from consideration for the end-to-end pipeline. Both the incumbent EC2 model and the recommended Hybrid model are technically feasible and reliable. However, the Hybrid model's use of declarative, containerized compute environments (like a Docker image for Fargate) makes it far more reproducible and less prone to configuration drift than a manually managed EC2 instance.

- **Total Cost of Ownership (TCO):** The TCO calculation must encompass both direct compute costs and indirect operational costs (i.e., engineering hours).
 - **EC2-Centric:** This model carries the highest TCO. It combines potentially inefficient, always-on or difficult-to-scale compute costs with the very high hidden cost of system administration for patching, security, and maintenance.
 - **Hybrid (Orchestrated Compute):** This model offers the lowest long-term TCO. It combines the consumption-based pricing of Snowflake and the orchestrator with highly efficient, ephemeral compute costs for the processing step (e.g., paying for Fargate resources only for the minutes they are running). Crucially, it dramatically reduces the operational burden on TxDOT's engineers, freeing them for higher-value work.
- **Scalability and Performance:** The Hybrid model provides a superior scalability story. The specialized processing step can be scaled independently (e.g., by running multiple Fargate tasks in parallel) to handle large influxes of GRIB2 files. Once the data is structured and loaded into Snowflake, TxDOT can leverage Snowflake's massively parallel processing (MPP) architecture and independently scalable virtual warehouses for all subsequent analytical queries, providing virtually unlimited performance.¹³ This is a significant advantage over the manually scaled EC2 model.
- **Maintainability and Operational Burden:** The Hybrid model is vastly more maintainable. Defining the entire GRIB2 processing environment within a single Dockerfile, managed in source control, is a modern, infrastructure-as-code practice. It is far superior to the legacy practice of manually configuring and maintaining a fleet of EC2 instances. The Snowflake platform itself requires zero maintenance from TxDOT.¹³
- **Security, Governance, and Data Lineage:** While the EC2 model creates a silo, the Hybrid model offers a clear and governable workflow. The processing step is a well-defined, isolated task within an orchestrated pipeline. The orchestrator (e.g., MWAA) provides a clear graph of the entire data flow, enhancing lineage tracking.²⁹ Once the data lands in Snowflake, it is immediately protected by Snowflake's comprehensive suite of security and governance features, including role-based access control, data masking, and audit trails.¹³

Final Recommendation for TxDOT

Based on the exhaustive analysis conducted in this report, the definitive recommendation is

for the Texas Department of Transportation to adopt the **Hybrid Cloud Pattern B: The Orchestrated Compute Architecture**.

This architecture is the only solution that meets all of TxDOT's requirements without significant compromise.

1. It **guarantees technical feasibility** by using the right tool for the job—dedicated AWS compute (Fargate or EC2)—for the complex, compiled GRIB2 decoding step.
2. It **maximizes cost-efficiency and scalability** by leveraging modern, cloud-native patterns like orchestration and ephemeral, serverless compute, ensuring resources are used and paid for only when needed.
3. It **drastically reduces the operational burden and TCO** compared to the incumbent EC2 model by abstracting away infrastructure management for both the processing environment and the data platform.
4. It **leverages the full power of the Snowflake Data Cloud** for everything downstream of the initial decoding, including scalable storage, world-class analytics, robust governance, and seamless data sharing.
5. It **avoids the unacceptable technical risk** of the "fully native" approach and the high operational drag and technical debt of the legacy EC2-centric model.

By implementing the Orchestrated Compute Hybrid Architecture, TxDOT can build a data pipeline that is robust, scalable, cost-effective, and maintainable, positioning its data infrastructure for future growth and innovation.

Works cited

1. pygrib-PyPI, accessed July 7, 2025, <https://pypi.org/project/pygrib/>
2. Installation — pygrib documentation - GitHub Pages, accessed July 7, 2025, <https://jswhit.github.io/pygrib/installing.html>
3. Installing pygrib Package - python - Stack Overflow, accessed July 7, 2025, <https://stackoverflow.com/questions/35640117/installing-pygrib-package>
4. wgrib2: grib2 utility - Climate Prediction Center, accessed July 7, 2025, <https://www.cpc.ncep.noaa.gov/products/wesley/wgrib2/>
5. wgrib2: how to compile - Climate Prediction Center, accessed July 7, 2025, https://www.cpc.ncep.noaa.gov/products/wesley/wgrib2/compile_questions.html
6. install_wgrib2: Install wgrib2. in rijaf-iri/CDT: Climate Data Tools - rdrv.io, accessed July 7, 2025, https://rdrv.io/github/rijaf-iri/CDT/man/install_wgrib2.html
7. Installation — wgrib2-docs documentation, accessed July 7, 2025, https://wgrib2-docs.readthedocs.io/en/latest/pywgrib2_s/pywgrib2_s_install.html
8. Creating an Amazon EC2 instance with remote root user access and configuring an usage alerting - DEV Community, accessed July 7, 2025, <https://dev.to/aws-builders/creating-an-amazon-ec2-instance-with-remote-root-user-access-and-configuring-an-usage-alerting-3ala>
9. Root access amazon ec2 - Server Fault, accessed July 7, 2025, <https://serverfault.com/questions/231658/root-access-amazon-ec2>
10. Enable root Access for Linux Instances - | Scalingbits, accessed July 7, 2025, <http://www.scalingbits.com/aws/sap/suse/rootaccess>

11. Getting Started with Snowpipe - Snowflake Quickstarts, accessed July 7, 2025, https://quickstarts.snowflake.com/guide/getting_started_with_snowpipe/index.html?index=..%2F.index&wttime&utm_campaign=na-us-en-&utm_source=website&utm_medium=partner&utm_content=-wb-pbs-demo-rudderstack&utm_cta=website-partner-accenture-learning-hub-snowflake-industry-day-how-networks-can-understand-customer-experience-and-improve-their-nps-and-user-retention-with-umlaut?wttime
12. Automating Snowpipe for Amazon S3 | Snowflake Documentation, accessed July 7, 2025, <https://docs.snowflake.com/en/user-guide/data-load-snowpipe-auto-s3>
13. Key Concepts & Architecture | Snowflake Documentation, accessed July 7, 2025, <https://docs.snowflake.com/en/user-guide/intro-key-concepts>
14. Snowpark API | Snowflake Documentation, accessed July 7, 2025, <https://docs.snowflake.com/en/developer-guide/snowpark/index>
15. How to Setup a Project in Snowpark Using a Python IDE - phData, accessed July 7, 2025, <https://www.phdata.io/blog/how-to-setup-a-project-in-snowpark-using-a-python-ide/>
16. Creating User-Defined Functions (UDFs) for DataFrames in Python, accessed July 7, 2025, <https://docs.snowflake.com/en/developer-guide/snowpark/python/creating-udfs>
17. User-defined functions overview - Snowflake Documentation, accessed July 7, 2025, <https://docs.snowflake.com/en/developer-guide/udf/udf-overview>
18. Using third-party packages | Snowflake Documentation, accessed July 7, 2025, <https://docs.snowflake.com/en/developer-guide/udf/python/udf-python-packages>
19. How to use third party Python packages not published on ..., accessed July 7, 2025, <https://community.snowflake.com/s/article/how-to-use-other-python-packages-in-snowpark>
20. Snowpark Python Packages in Snowflake Conda Channel, accessed July 7, 2025, <https://snowpark-python-packages.streamlit.app/>
21. Access PyPI Packages in Snowpark via UDFs and Stored Procedures, accessed July 7, 2025, <https://www.snowflake.com/en/blog/snowpark-supports-pypi-packages/>
22. Python UDF limitations | Snowflake Documentation, accessed July 7, 2025, <https://docs.snowflake.com/en/developer-guide/udf/python/udf-python-limitations>
23. A Definitive Guide to Python UDFs in the Snowflake UI - InterWorks, accessed July 7, 2025, <https://interworks.com/blog/2022/08/09/definitive-guide-python-udfs-snowflake-ui/>
24. snowflake-snowpark-python - PyPI, accessed July 7, 2025, <https://pypi.org/project/snowflake-snowpark-python/>
25. Getting Started With External Functions on AWS - Snowflake Quickstarts, accessed July 7, 2025,

https://quickstarts.snowflake.com/guide/getting_started_external_functions_aws/index.html?index=..%2F..index

26. Introduction to external functions - Snowflake Documentation, accessed July 7, 2025,
<https://docs.snowflake.com/en/sql-reference/external-functions-introduction>
27. What Are Snowflake External Functions? - Secoda, accessed July 7, 2025,
<https://www.secoda.co/learn/what-are-snowflake-external-functions>
28. Data Enrichment using Snowflake External Functions | by Balkaran Brar - Medium, accessed July 7, 2025,
<https://balkaranbrar.medium.com/data-enrichment-using-snowflake-external-functions-49c32f31ff45>
29. Use Snowflake with Amazon MWAA to orchestrate data pipelines ..., accessed July 7, 2025,
<https://aws.amazon.com/blogs/big-data/use-snowflake-with-amazon-mwaa-to-orchestrate-data-pipelines/>